LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

Apache Log4j2 Issue #4255: FilteredObjectInputStream Deserialization Exposure

Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-08-26
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Threat Intelligence Report

Published: 2026-08-26 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

Apache Log4j2 Issue #4255: FilteredObjectInputStream Deserialization Exposure

1. Executive Summary

BLUF: Apache logging-log4j2 issue #4255 is a same-day public exposure candidate concerning a claimed FilteredObjectInputStream allowlist bypass through java.rmi.MarshalledObject. If validated in a vulnerable architecture, the issue could permit unfiltered Java deserialization in services that receive serialized LogEvent objects over the network. Lost Boys Cyber assesses this as a high-priority triage item for environments running native Java serialized Log4j receivers, but not as a universal Log4j RCE and not as an Apache-confirmed CVE at publication time.

The key operational distinction is scope. Ordinary applications that use Log4j2 only to write local files, console logs, JSON logs, or standard syslog output are not directly exposed to this specific issue. The exposure pivots are receiver-side components and custom code paths that deserialize Java LogEvent objects from a network socket, such as ObjectInputStreamLogEventBridge-like services, legacy TcpSocketServer serialized receiver paths, or third-party products that embedded similar receiver logic.

The public record is unusual: the GitHub issue exists, was opened on 2026-08-24, was closed on 2026-08-26, and the issue body was unavailable via GitHub API during review. Same-day secondary reporting and GitHub comments reference archived material and multiple PoC repositories. Because no CVE or Apache security advisory was observed, defenders should treat this as an emerging vulnerability candidate: move quickly on exposure discovery and compensating controls, but label external reporting and executive messaging with appropriate uncertainty.

2. Vulnerability Metadata

Field	Current Assessment
Tracking name	Apache logging-log4j2 issue #4255 / FilteredObjectInputStream allowlist bypass via MarshalledObject
CVE status	No CVE observed at report creation time
Vendor advisory status	No Apache security advisory observed at report creation time
Issue status	GitHub issue #4255 closed; original body unavailable via GitHub API
Public disclosure date	Secondary article reviewed was published 2026-08-26T14:24:49Z; issue was created 2026-08-24T07:18:38Z
Claimed affected components	log4j-api FilteredObjectInputStream / SerializationUtil and log4j-core LogEventProxy serialized event handling
Claimed affected versions	Secondary reporting claims log4j-api 2.11.0 through 2.26.1 and log4j-core 2.8.0 through 2.26.1
Exploit class	Java native deserialization / CWE-502-style exposure
Practical severity	High to Critical only where a native serialized LogEvent receiver is reachable and a usable gadget chain exists
Default exposure	Not universal; requires receiver architecture, not normal log-writing use
Primary defensive action	Disable, segment, or migrate native Java serialized LogEvent receivers; monitor for official Apache/CVE follow-up

3. Source Review & Confidence

Source	What It Proves	What It Does Not Prove	Confidence
Apache logging-log4j2 issue #4255 metadata via GitHub API	Issue existed, was closed, timestamps, comments, and community links	Original technical body, Apache acceptance, CVE assignment, or patch status	Medium
Issue comments	Community reaction and links to archive/PoC repositories	Correctness of exploit claims or production exploitation	Medium-Low
The CyberSec Guru same-day article	Detailed secondary description of claimed root cause, scope, affected versions, and mitigations	Vendor confirmation or independent victim telemetry	Medium
SecurityOnline and similar secondary reporting	Corroboration that the claim is being redistributed publicly	Authoritative vulnerability status	Medium-Low
Apache Logging security page and CycloneDX VDR	Absence of an observed official Apache advisory for #4255 during review	Future status; these pages may update after publication	High for review time, time-sensitive
Public PoC repository references	PoC material is circulating	Safety, correctness, exploitability across real environments	Medium-Low

Analyst confidence is medium that the issue is a real exposure candidate that defenders should triage immediately. Confidence is lower that it will remain valid exactly as described in secondary reporting because the original issue text was unavailable and Apache had not published a matching security advisory or CVE at review time. Confidence is high that the issue is not equivalent to Log4Shell and should not be messaged as a universal log-string-triggered RCE.

4. Vulnerability Details

4.1. Claimed root cause

Secondary reporting describes a bypass in Log4j2's FilteredObjectInputStream class filtering model. FilteredObjectInputStream reportedly uses a class-name allowlist during resolveClass() to prevent unsafe Java deserialization. The claimed blind spot is java.rmi.MarshalledObject: the outer MarshalledObject can pass an allowlist check while carrying an opaque serialized inner object in its byte payload. If later code calls MarshalledObject.get(), Java can deserialize the inner payload through a separate ObjectInputStream path that is not constrained by the original FilteredObjectInputStream allowlist.

4.2. Claimed auto-trigger path

The issue is tied to serialized LogEvent handling. Secondary reporting states that Log4j's LogEventProxy serialized-event reconstruction may call into the marshalled message path while rebuilding a LogEvent. If an attacker can send a crafted serialized LogEvent to a receiver that deserializes network-supplied Java objects, the receiver may unwrap the MarshalledObject and deserialize attacker-controlled content in the receiver process.

4.3. Scope limitation

This is a receiver-side Java serialization exposure, not a normal logging-string bug. An application that merely includes Log4j2 and writes logs locally is not enough. A plausible vulnerable deployment needs at least:

- A service that accepts serialized Java LogEvent objects from the network.
- Log4j2 versions and receiver code matching the claimed affected paths.
- Network reachability from an attacker or compromised host.
- A classpath with a usable deserialization gadget chain for RCE, or enough object-graph complexity for DoS.
- Insufficient network segmentation, authentication, or JVM-level serialization filtering.

4.4. Why defenders should still prioritize it

Even narrow Java deserialization issues can be severe when receiver services are exposed. A serialized log receiver is often assumed to be infrastructure plumbing and may be overlooked by internet exposure reviews. If a receiver is reachable from an untrusted network and runs with broad application permissions, the impact can exceed the logging layer and become process-level execution in the Java service context.

5. Attack Scenarios

Scenario	Preconditions	Attacker Action at a High Level	Likely Outcome	Defender Priority
Exposed serialized log receiver RCE	Native Java serialized LogEvent receiver reachable from untrusted network and gadget chain on classpath	Send crafted Java serialized stream to receiver	Possible code execution in Java receiver process	Emergency isolation and receiver shutdown/migration
Internal pivot to logging collector	Receiver not internet-facing but reachable from compromised internal host	Abuse trusted internal log-forwarding path	Lateral movement or collector compromise	Network ACLs and internal east-west monitoring
Object-graph bomb DoS	Receiver accepts Java serialization but lacks gadget chain suitable for RCE	Send resource-intensive serialized object graph	CPU/memory exhaustion or receiver instability	Rate limiting, JVM filters, process/resource monitoring
Log injection / telemetry poisoning	Receiver accepts unauthenticated serialized log events	Submit attacker-controlled log records	False records, SIEM noise, or IR confusion	Authenticate log sources and correlate receiver inputs
Opportunistic scanning	Public PoC references circulate	Probe common or discovered Java listener ports	Exposure discovery, potential automated exploitation attempts	Hunt Java serialization stream markers and Java child-process anomalies

6. Threat Landscape & Exploitation Status

Public attention is active because the GitHub issue comments reference archives and PoC repositories. That does not automatically prove mass exploitation or vendor-confirmed criticality. At review time, Lost Boys Cyber observed evidence of public discussion and PoC circulation, but no authoritative Apache advisory, no CVE, and no credible victim reporting.

The likely near-term threat is opportunistic exposure hunting. Attackers and researchers may scan for Java services that accept native serialized streams, especially where product documentation, default ports, or code strings disclose ObjectInputStreamLogEventBridge, TcpSocketServer, or SerializedLayout-style usage. Internal enterprise risk is highest in older Java monitoring stacks, custom logging aggregators, copied sample receiver code, lab-to-production tooling, and third-party analytics components that embed serialized Log4j receiver behavior.

7. Affected Products and Exposure

Exposure Area	What to Inventory	Risk Signal	Recommended Disposition
Java services	Runtime Java processes and listening ports	Java process accepting inbound TCP from non-local sources	Review purpose, owner, network ACLs, and protocol
Log4j receiver code	ObjectInputStreamLogEventBridge, TcpSocketServer, createSerializedSocketServer,	Code path accepts serialized LogEvent objects	Disable or isolate until vendor guidance is clear

	SerializedLayout, LogEventBridge references		
Dependencies	log4j-api and log4j-core versions named in secondary reporting	Claimed affected versions in runtime SBOM or shaded JARs	Track for future CVE; do not rely on version-only risk without receiver exposure
Gadget chain libraries	commons-collections, commons-beanutils, groovy, older Spring/InvokerTransformer-era libraries	Old gadget-capable dependencies in receiver classpath	Remove/upgrade libraries; run SCA across receiver deployment
Network perimeter	Public or DMZ access to Java receiver ports	Unknown Java listener reachable externally	Block immediately unless explicitly required and authenticated
Internal segmentation	Collector reachable broadly from workstation/server VLANs	Many-to-one trust to logging collector	Restrict to expected log forwarders only

8. Mitigation and Workarounds

Priority	Action	Notes
P0	Identify and disable native Java serialized LogEvent receivers exposed to untrusted networks	This is the exposure condition that matters most; normal local logging is not enough
P0	Add network ACLs around any receiver that cannot be disabled immediately	Allow only known log forwarders; block internet/DMZ and broad east-west access
P1	Migrate log transport away from Java native serialization	Prefer JSON, RFC5424/syslog, OTLP, or another structured format over authenticated TLS
P1	Inventory log4j-api/log4j-core versions in services that actually receive serialized events	Version presence alone is not sufficient; prioritize receiver-side deployments
P1	Remove or upgrade known Java deserialization gadget libraries in receiver classpaths	Reduces RCE reliability but does not remove deserialization risk
P1	Evaluate JVM-level serial filters on JDK 9+ where compatible	A filter rejecting java.rmi.MarshalledObject may break legitimate serialized LogEvent forwarding; test before broad deployment
P2	Add alerts for Java listener exposure, Java serialization stream markers, and Java child-process anomalies	Detect scanning and potential exploitation attempts
P2	Monitor Apache security page, Apache VDR, Maven release notes, and issue references for official CVE/patch updates	Advisory status may change after publication

9. Detection Engineering

9.1. Detection Summary

Signal	Telemetry Source	Analytic Goal
Java process with unusual listening port	EDR socket telemetry, netstat/ss inventory, CMDB	Discover receiver services that may deserialize network input
ObjectInputStreamLogEventBridge / TcpSocketServer strings in repo or config	Source search, container filesystem, package inventory	Identify code/config exposure pivots
Java serialization magic bytes ac ed 00 05 to Java service	Zeek/Suricata/packet metadata	Detect probing or native serialization traffic
Java spawning shell/network utilities	EDR process telemetry	Detect possible post-deserialization execution
Sudden Java CPU/memory spike after inbound network payload	EDR/resource telemetry	Detect object-graph bomb DoS attempts

```
title: Java Process Spawns Shell Or Network Tool After Possible Deserialization
id: 42550001-2026-0826-log4j-serialized-receiver-child-process
status: experimental
description: Detects Java processes spawning shells or transfer tools, a generic post-deserialization
execution signal relevant to Log4j2 issue #4255-style receiver exposure.
references:
  - https://github.com/apache/logging-log4j2/issues/4255
  - https://thecybersecguru.com/news/log4j2-deserialization-vulnerability-rce/
author: Lost Boys Cyber
date: 2026/08/26
logsource:
  product: windows
  category: process_creation
detection:
  parent_java:
    ParentImage|endswith:
      - '\java.exe'
      - '\javaw.exe'
  child_suspicious:
    Image|endswith:
      - '\cmd.exe'
      - '\powershell.exe'
      - '\pwsh.exe'
      - '\wscript.exe'
      - '\cscript.exe'
      - '\curl.exe'
      - '\wget.exe'
      - '\certutil.exe'
  condition: parent_java and child_suspicious
falsepositives:
  - Build servers, application updaters, and legitimate Java-based automation may launch shells; scope
to logging collectors and unexpected listener services.
level: high
```

```
// Microsoft Defender: find Java processes exposing likely serialized logging receiver ports or unusual
listeners
DeviceNetworkEvents
| where Timestamp > ago(14d)
| where InitiatingProcessFileName in~ ("java.exe", "java", "javaw.exe")
| where ActionType has_any ("Listening", "ConnectionSuccess", "InboundConnectionAccepted")
| extend PortOfInterest = LocalPort in (4560, 4561, 4562, 4563, 4712, 4713, 5000) or RemotePort in
(4560, 4561, 4562, 4563, 4712, 4713, 5000)
| where PortOfInterest or InitiatingProcessCommandLine has_any ("ObjectInputStreamLogEventBridge",
```

```
"TcpSocketServer", "SerializedLayout", "log4j")
| project Timestamp, DeviceName, InitiatingProcessFileName, InitiatingProcessCommandLine, LocalIP,
LocalPort, RemoteIP, RemotePort, ActionType
```

```
// Microsoft Defender: Java process child execution hunt
DeviceProcessEvents
| where Timestamp > ago(14d)
| where InitiatingProcessFileName in~ ("java.exe", "javaw.exe", "java")
| where FileName in~ ("cmd.exe", "powershell.exe", "pwsh.exe", "wscript.exe", "cscript.exe",
"curl.exe", "wget.exe", "certutil.exe", "bash", "sh", "nc", "ncat")
| project Timestamp, DeviceName, InitiatingProcessCommandLine, FileName, ProcessCommandLine,
AccountName, ReportId
```

```
index=* (process_name=java OR process_name=java.exe OR parent_process_name=java.exe OR
parent_process_name=java)
("ObjectInputStreamLogEventBridge" OR "TcpSocketServer" OR "SerializedLayout" OR
"FilteredObjectInputStream" OR "MarshalledObject" OR "AC ED 00 05")
| stats count min(_time) as first_seen max(_time) as last_seen values(host) as hosts values(source) as
sources by process_name parent_process_name
| convert ctime(first_seen) ctime(last_seen)
```

```
alert tcp any any -> $HOME_NET any (msg:"POSSIBLE Java serialization stream to internal service -
Log4j2 issue 4255 triage"; flow:to_server,established; content:"|AC ED 00 05|"; depth:4;
classtype:attempted-recon; sid:4255001; rev:1; metadata: created_at 2026_08_26, attack_target Server,
confidence Medium;)
```

10. Threat Hunting

10.1. Hunt Hypotheses

Hypothesis	Rationale	Primary Data
A serialized Log4j receiver is exposed but undocumented	Receiver services may be inherited from samples or older logging infrastructure	EDR network telemetry, source search, container inventory
A public PoC was used to probe a Java listener	Comments and secondary sources show public PoC circulation	Packet metadata, firewall logs, Java listener logs
A deserialization attempt executed a child process from Java	Gadget-chain exploitation often results in shell or network utility execution	EDR process trees
A DoS attempt targeted a Java receiver without a working gadget chain	Object graph payloads may cause CPU/memory pressure	Resource telemetry, JVM metrics, crash/restart logs

```
# Defensive repository/configuration triage for issue #4255 exposure pivots.
# Run against source repos or unpacked application/container filesystems you are authorized to assess.
root="${1:-.}"
find "$root" -type f \( -name '*.java' -o -name '*.xml' -o -name '*.properties' -o -name '*.yaml' -o
-name '*.yml' -o -name 'pom.xml' -o -name 'build.gradle' -o -name '*.jar' \) 2>/dev/null |
while read -r path; do
  case "$path" in
    *.jar)
      case "$path" in
        *log4j-api-2.*.jar|*log4j-core-2.*.jar|*commons-collections-3.2.1*.jar|*commons-beanutils*.jar|
*groovy*.jar)
          printf 'JAR_REVIEW,%s\n' "$path" ;;
      esac ;;
    *)
      if grep -Eiq 'ObjectInputStreamLogEventBridge|TcpSocketServer|createSerializedSocketServer|
SerializedLayout|FilteredObjectInputStream|MarshalledObject|LogEventProxy' "$path"; then
        printf 'SOURCE_OR_CONFIG_PIVOT,%s\n' "$path"
        grep -Ein 'ObjectInputStreamLogEventBridge|TcpSocketServer|createSerializedSocketServer|
SerializedLayout|FilteredObjectInputStream|MarshalledObject|LogEventProxy' "$path" | head -20
      fi ;;
  esac
done
```

```
// Correlate inbound connections to Java listeners with suspicious Java child processes on the same
```

```
host.
let JavaNetwork = DeviceNetworkEvents
| where Timestamp > ago(14d)
| where InitiatingProcessFileName in~ ("java.exe", "javaw.exe", "java")
| where ActionType has_any ("ConnectionSuccess", "InboundConnectionAccepted")
| project NetTime=Timestamp, DeviceName, LocalIP, LocalPort, RemoteIP, RemotePort,
NetCmd=InitiatingProcessCommandLine;
let JavaChildren = DeviceProcessEvents
| where Timestamp > ago(14d)
| where InitiatingProcessFileName in~ ("java.exe", "javaw.exe", "java")
| where FileName in~ ("cmd.exe", "powershell.exe", "pwsh.exe", "bash", "sh", "curl.exe", "wget.exe",
"nc", "ncat")
| project ProcTime=Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessCommandLine;
JavaNetwork
| join kind=innerunique JavaChildren on DeviceName
| where ProcTime between (NetTime .. NetTime + 10m)
| project NetTime, ProcTime, DeviceName, LocalIP, LocalPort, RemoteIP, RemotePort, FileName,
ProcessCommandLine, NetCmd
```

11. MITRE ATT&CK Mapping

Technique	Name	Why It Applies
T1190	Exploit Public-Facing Application	A reachable serialized LogEvent receiver is the likely initial access surface
T1059	Command and Scripting Interpreter	Successful deserialization gadgets may execute shells or scripting engines
T1105	Ingress Tool Transfer	Post-exploitation child processes may fetch second-stage tooling
T1499	Endpoint Denial of Service	Object-graph bombs may exhaust receiver CPU/memory
T1562.008	Impair Defenses: Disable or Modify Cloud Logs	Abuse of logging receivers can poison, confuse, or impair telemetry

12. Validation / Lab Testing

Validation should be defensive and non-production. The objective is to confirm whether the organization has the exposure architecture, not to reproduce weaponized payloads.

Test	Method	Safe Expected Result
Receiver inventory	Search source, configs, containers, and runtime processes for ObjectInputStreamLogEventBridge, TcpSocketServer, SerializedLayout, and LogEventProxy references	No native Java serialized LogEvent receivers, or receivers documented and segmented
Network exposure review	Inspect firewall rules, EDR socket telemetry, and service discovery for Java listener ports	No untrusted access to serialized receivers
Dependency review	Generate SBOM for receiver services and inspect log4j-api/log4j-core plus gadget-capable libraries	Receiver dependencies upgraded and gadget libraries removed where possible
JVM filter staging test	On JDK 9+, apply candidate serial filters in	Logging still works if serialization is

	staging only	unused; serialized forwarding breaks visibly if dependent
Telemetry validation	Trigger benign Java child-process and listener test events in a lab	Detection queries produce expected alerts without production risk

13. Recommended Actions Summary

Timeframe	Owner	Action
Today	Platform / AppSec	Inventory all Java services that listen for inbound TCP and all source/config references to serialized Log4j receiver classes
Today	Infrastructure	Block public and broad east-west access to any native Java serialized LogEvent receiver
Today	Engineering	Determine whether Log4j2 is used for receiver-side Java serialization or only local/log-output paths
24 hours	Detection Engineering	Deploy Java listener, Java child-process, and serialization stream detection content with tuning
24-72 hours	Engineering	Remove or upgrade gadget-prone dependencies from any receiver classpath
72 hours	Architecture	Migrate serialized log forwarding to JSON/syslog/OTLP or another non-native-serialization transport over authenticated TLS
Ongoing	Threat Intel / AppSec	Monitor Apache issue #4255, Apache security advisories, Apache VDR, Maven release notes, and CVE feeds for official assignment or patch

14. Indicators, Observables, and Source Limitations

No victim-specific malicious infrastructure, file hashes, domains, or IP addresses were identified in reviewed sources. Operational indicators are configuration, protocol, and behavior based.

Observable	Type	Operational Use
ObjectInputStreamLogEventBridge	Code/config string	Potential serialized LogEvent receiver exposure
TcpSocketServer / createSerializedSocketServer	Code/config string	Legacy serialized receiver path requiring triage
SerializedLayout	Code/config string	Possible native serialized logging transport usage

FilteredObjectInputStream / MarshalledObject	Code/class string	Claimed issue #4255 root-cause components
AC ED 00 05	Protocol bytes	Java serialization stream magic in authorized network telemetry
Java listener ports 4560-4563, 4712-4713, 5000 or custom	Network observable	Candidate receiver discovery; validate locally before assuming vulnerability
Java spawning cmd, powershell, bash, sh, curl, wget, nc, or ncat	Behavior	Possible post-deserialization execution or suspicious automation
Sudden Java CPU/memory pressure after inbound traffic	Behavior	Possible object-graph DoS attempt

15. References

- [1] Apache logging-log4j2 issue #4255: https://github.com/apache/logging-log4j2/issues/4255
- [2] The CyberSec Guru, New Log4j2 Flaw Could Enable Remote Code Execution Through Java Deserialization: https://thecybersecguru.com/news/log4j2-deserialization-vulnerability-rce/
- [3] SecurityOnline, Unpatched Log4j RCE Flaw Publicly Disclosed: https://securityonline.info/log4j-filteredobjectinputstream-rce/
- [4] Apache Logging Services Security: https://logging.apache.org/security.html
- [5] Apache CycloneDX Vulnerability Disclosure Report: https://logging.apache.org/cyclonedx/vdr.xml
- [6] GitHub issue comment referenced PoC: https://github.com/dinosn/log4j-4255
- [7] GitHub issue comment referenced PoC: https://github.com/joanbono/log4j-4255-exploit
- [8] GitHub issue comment referenced PoC: https://github.com/hypnguyen1209/log4j2-rce